

Recipe for baking a GCC

Ingredients and community of the GNU Compiler Collection, 2025

Thomas Schwinge <tschwinge@baylibre.com>

Kernel Recipes 2025, Paris, 2025-09-23

I'm a Linux kernel developer, too! ;-D

https://git.kernel.org/pub/scm/

index : kernel/git/torvalds/linux.git

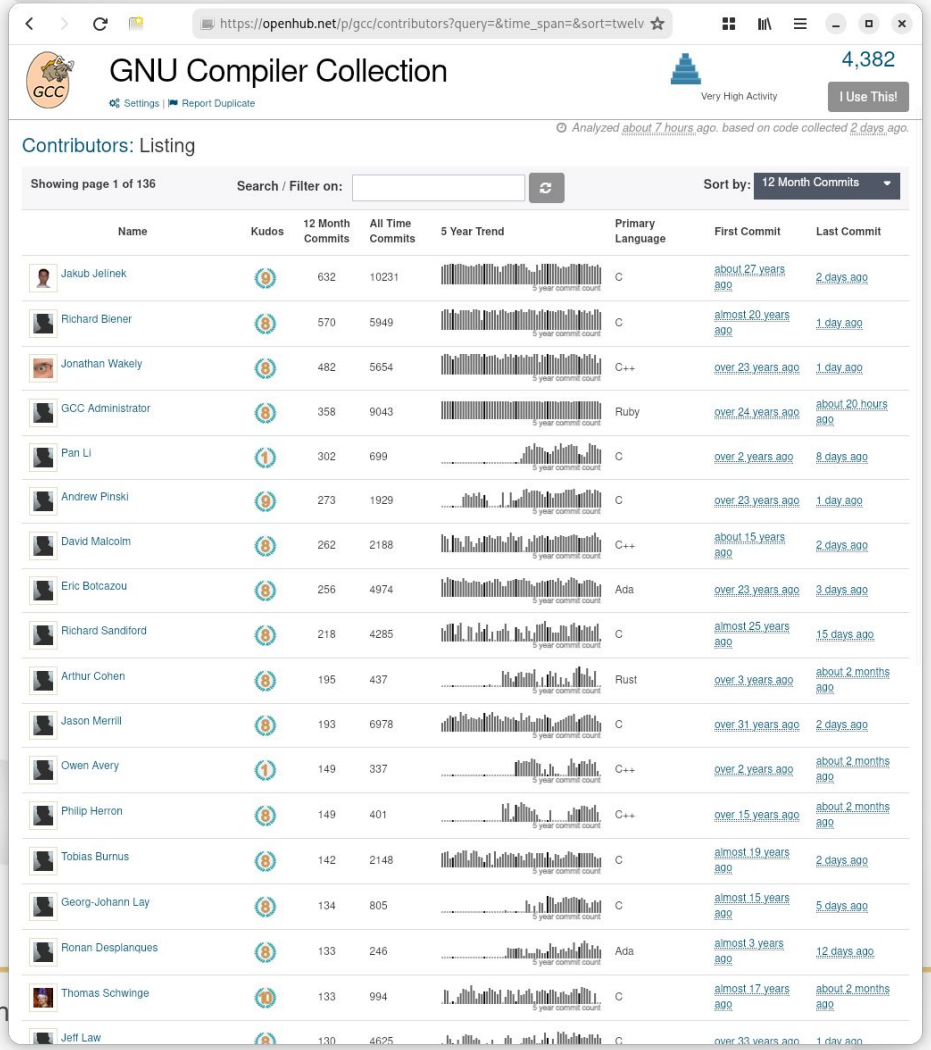
Linux kernel source tree

about summary refs log tree commit diff stats author Thomas Sch

Age	Commit message (Expand)	Author	Files	Lines
2013-01-11	sh: Fix FDPIC binary loader	Thomas Schwinge	1	-2/+2
2012-03-28	SH: pci-sh7780: enable big-endian operation.	Thomas Schwinge	1	-3/+12
2012-03-07	sh: fix up the ubc clock definition for sh7785.	Thomas Schwinge	1	-1/+1

generated by cgit 1.2.3-korg (git 2.43.0) at 2025-09-19 15:49:10 +0000





https://openhub.net/p/gcc/contributors?query=&time_span=&sort=twelv

GNU Compiler Collection

Settings | Report Duplicate

Very High Activity

4,382

I Use This!

Analyzed about 7 hours ago. based on code collected 2 days ago.

Contributors: Listing

Showing page 1 of 136

Search / Filter on:

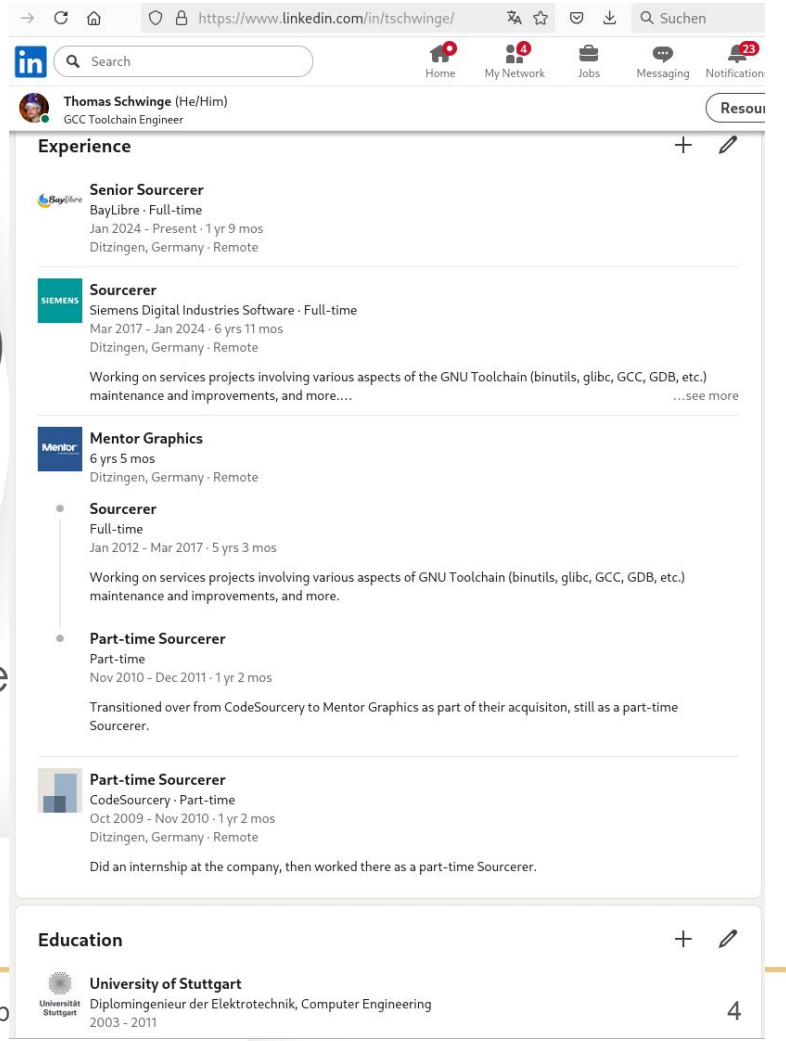
Sort by: 12 Month Commits

Name	Kudos	12 Month Commits	All Time Commits	5 Year Trend	Primary Language	First Commit	Last Commit
Jakub Jelinek	9	632	10231	5 year commit count	C	about 27 years ago	2 days ago
Richard Biener	8	570	5949	5 year commit count	C	almost 20 years ago	1 day ago
Jonathan Wakely	8	482	5654	5 year commit count	C++	over 23 years ago	1 day ago
GCC Administrator	8	358	9043	5 year commit count	Ruby	over 24 years ago	about 20 hours ago
Pan Li	1	302	699	5 year commit count	C	over 2 years ago	8 days ago
Andrew Pinski	9	273	1929	5 year commit count	C	over 23 years ago	1 day ago
David Malcolm	8	262	2188	5 year commit count	C++	about 15 years ago	2 days ago
Eric Botcazou	8	256	4974	5 year commit count	Ada	over 23 years ago	3 days ago
Richard Sandiford	8	218	4285	5 year commit count	C	almost 25 years ago	15 days ago
Arthur Cohen	8	195	437	5 year commit count	Rust	over 3 years ago	about 2 months ago
Jason Merrill	8	193	6978	5 year commit count	C	over 31 years ago	2 days ago
Owen Avery	1	149	337	5 year commit count	C++	over 2 years ago	about 2 months ago
Philip Herron	8	149	401	5 year commit count	C++	over 15 years ago	about 2 months ago
Tobias Burnus	8	142	2148	5 year commit count	C	almost 19 years ago	2 days ago
Georg-Johann Lay	8	134	805	5 year commit count	C	almost 15 years ago	5 days ago
Ronan Desplanques	8	133	246	5 year commit count	Ada	almost 3 years ago	12 days ago
Thomas Schwinge	10	133	994	5 year commit count	C	almost 17 years ago	about 2 months ago
Jeff Law	8	130	4625	5 year commit count	C	over 33 years ago	1 day ago



“Me” (some of it, anyway)

- Background in electronics
- Commodore 64 (from flea market, ~1992)
- PC, Intel Pentium 60 (~1994)
- User of Open Source/Free Software since ~1997
- Linux 2.0.36 (1998)
- Developer of Open Source/Free Software since ~2001
  - Quickly got into systems software
- Internship at CodeSourcery, 2009



CodeSourcery -> BayLibre

BayLibre, <https://baylibre.com/>
Founded 2013
France and USA, with engineers world-wide
Linux kernel development and related: Android, firmware, uboot, Yocto, Zephyr, MbedTLS, …

Compiler Services Team
1997 – **CodeSourcery** founded
(I joined 2009)
2010 – CS acquired by **Mentor Graphics**
2017 – Mentor acquired by **Siemens** (Siemens Digital Industries Software)
Q4/2023 – Siemens stops open-source work related to embedded + compiler services
Q1/**2024** – **BayLibre** gained a new business unit when part of the compiler-services team joined
<https://baylibre.com/baylibre-expands-open-source-business-to-include-compiler-services/>



Focus of the Compiler Services Team at BayLibre

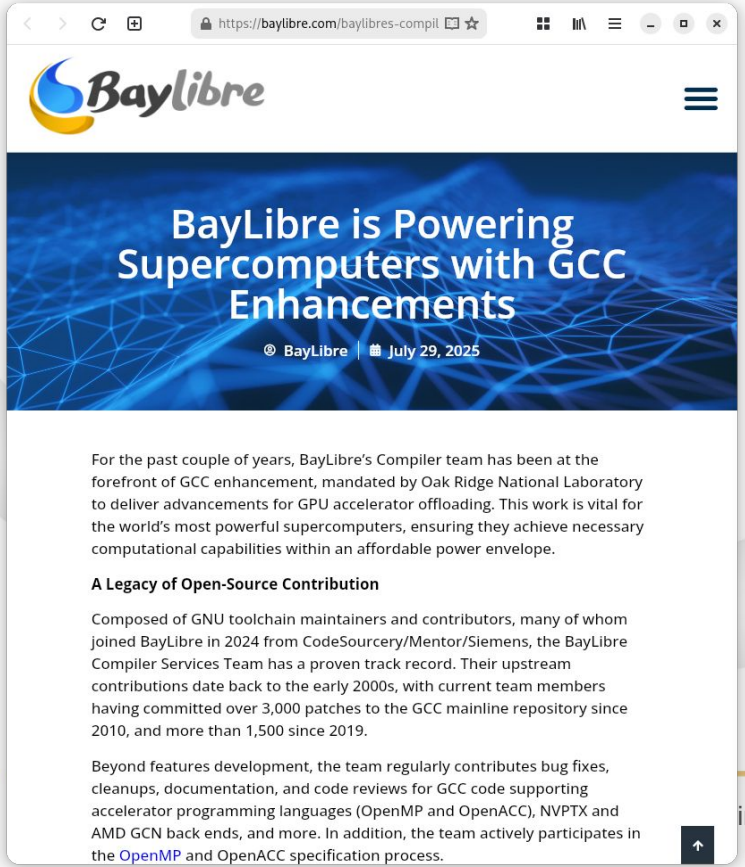
GCC/GNU Toolchain[1)] development of all kinds

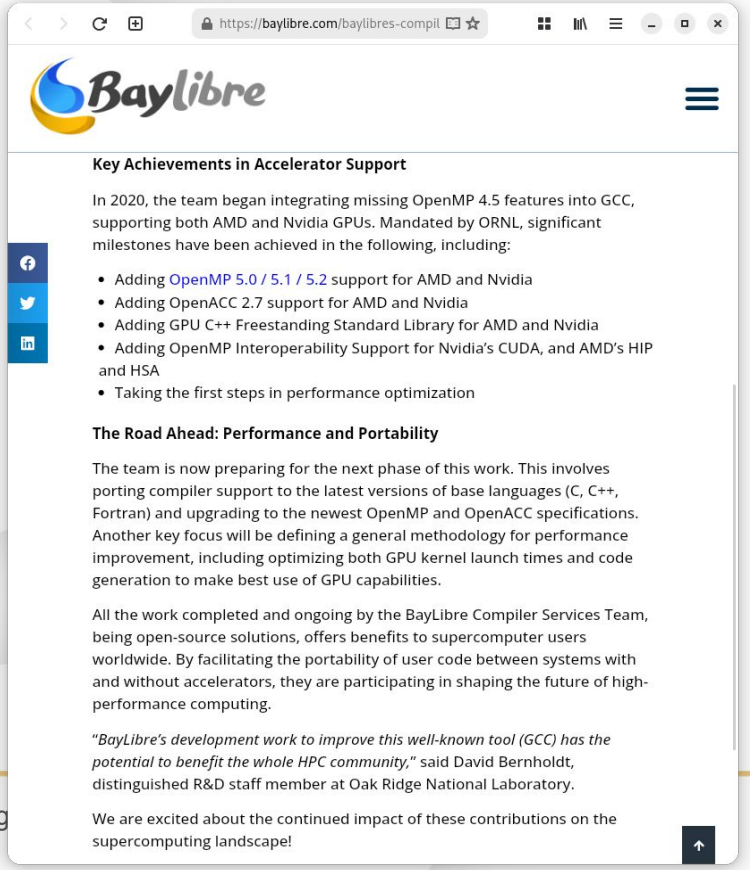
- Tiny embedded (RISC-V, for example)
- High-performance computing, GPU code offloading
- Everything in between
- Weird host support ("Windows")



<https://www.olcf.ornl.gov/olcf-resources/compute-systems/frontier/>

Focus of the Compiler Services Team at BayLibre





<https://baylibre.com/baylibres-compiler-services-gcc-enhancements/>

Back to GCC itself

I try to be objective, but still: the following is my interpretation, has not been blessed by “GCC Management”, GNU, FSF.



https://gcc.gnu.org/

GCC, the GNU Compiler Collection



The GNU Compiler Collection includes front ends for C, C++, Objective-C, Fortran, Ada, Go, D, Modula-2, and COBOL as well as libraries for these languages (libstdc++,...). GCC was originally written as the compiler for the GNU operating system. The GNU system was developed to be 100% free software, free in the sense that it respects the user's freedom.

We strive to provide regular, high quality releases, which we want to work well on a variety of native and cross targets (including GNU/Linux), and encourage everyone to contribute changes or help testing GCC. Our sources are readily and freely available via Git and weekly snapshots.

Major decisions about GCC are made by the steering committee, guided by the mission statement.

News

GCC 15.2 released [2025-08-08]

GNU Tools Cauldron 2025 [2025-08-01]
Porto, Portugal, September 26-28 2025

GCC 12.5 released [2025-07-11]

GCC 13.4 released [2025-06-05]

GCC 14.3 released [2025-05-23]

GCC 15.1 released [2025-04-25]

COBOL front end added [2025-04-17]
The COBOL programming language front end has been added to GCC. This front end was contributed by COBOLworx.

GCC developer room at FOSDEM 2025: Call for Participation open [2024-10-30]
FOSDEM 2025: Brussels, Belgium, February 1-2 2025

Supported Releases

GCC 15.2 (changes)
Status: 2025-08-08 (regression fixes & docs only).
Serious regressions. All regressions.

GCC 14.3 (changes)
Status: 2025-05-23 (regression fixes & docs only).
Serious regressions. All regressions.

GCC 13.4 (changes)
Status: 2025-06-05 (regression fixes & docs only).
Serious regressions. All regressions.

Development: GCC 16.0 (release criteria, changes)
Status: 2025-04-17 (general development).
Serious regressions. All regressions.

Search our site

About GCC
Mission
Releases
Snapshots
Mailing lists
Contributors
IRC
@gnutools
Mastodon
Make A Donation

Documentation
Installation
· Platforms
Manual
FAQ
Wiki
Pointers

Download
Mirrors
Binaries

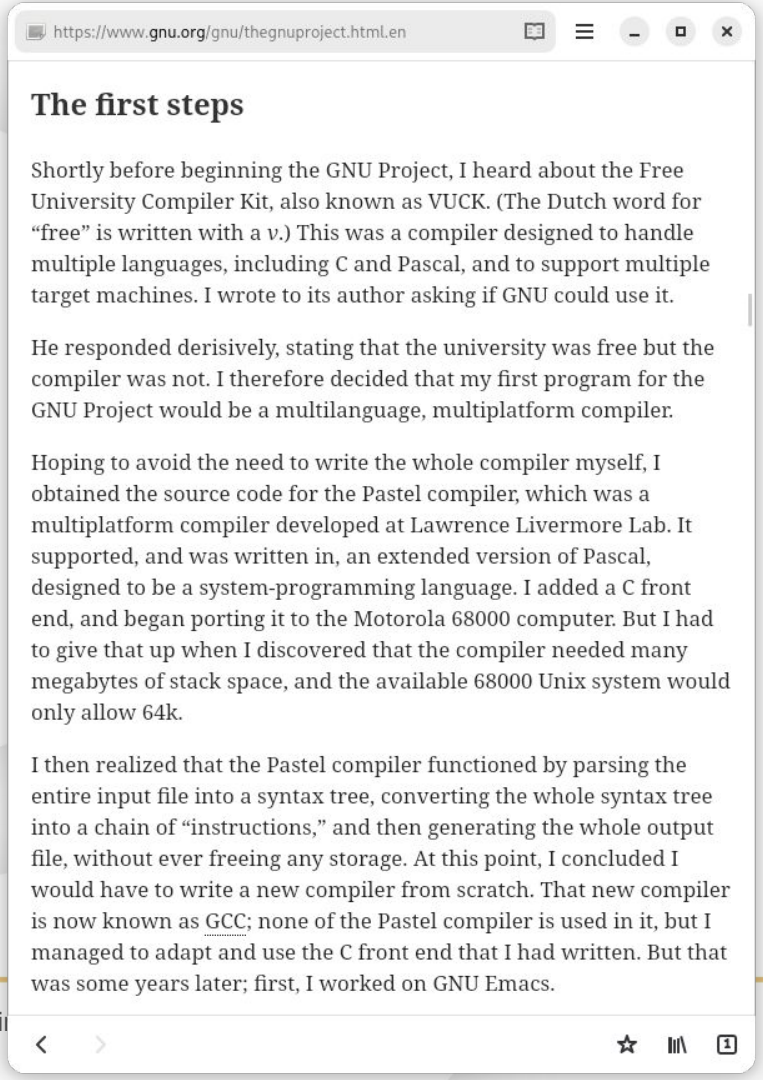
Sources
Git
...write access
Rsync

Development
Plan & Timeline
Contributing
Why contribute?
Open projects
Front ends
Back ends
Extensions
Testing
Benchmarks



GNU/GCC history







GCC first released in 1987

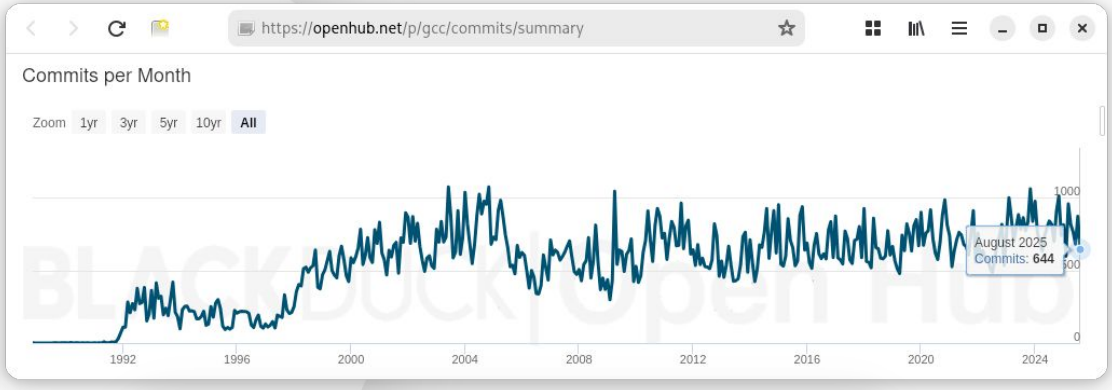


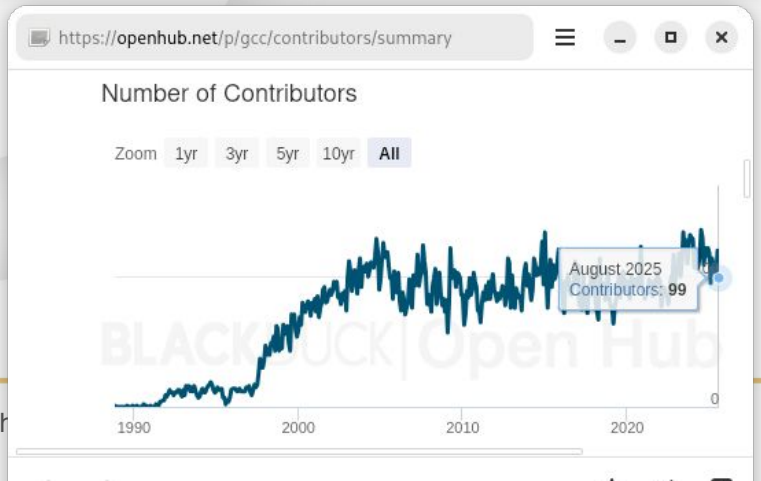




GCC still going strong







Who contributes to GCC?

For example, GCC 15 release (~1 y), number of commits by domain:

1. 1876 redhat.com
2. 1064 adacore.com
3. 675 gmail.com (66 persons)
4. 573 arm.com
5. 540 intel.com
6. 490 suse.de (+ 80 suse.cz, see below)
7. 406 gcc.gnu.org (22 persons)
8. 395 baylibre.com
9. 387 embecosm.com
10. 245 quicinc.com

11. 236 ventanamicro.com
12. 152 linux.ibm.com
13. 149 gjlay.de (1 person)
14. 104 nvidia.com
15. 99 gentoo.org
16. 91 linaro.org
17. 88 googlemail.com
18. 84 sandoe.co.uk (1 person)
19. 80 suse.cz
20. 75 rivosinc.com

⋮ …

Data prepared by my colleague Tobias Burnus, 2025-04–17

GCC source code

- Git repository: <https://gcc.gnu.org/git/gcc.git>
  - ALL history got converted (minimal repairs), including non-trivial branch structure
    - Effort by GCC people plus Eric S. Raymond
- Before 2020: Subversion
  - A good number of developers used ‘git-svn’
- Before 2006: CVS
- Before that: probaby RCS, Emacs versioned backup files?



GCC structure, “source code”

- Most of it is “genuine GCC code”
- Also some code imported from/shared with others (may be incomplete):
  - ‘gcc/ada/’
  - ‘gcc/d/dmd/’, ‘libphobos/’
  - ‘gcc/go/gofrontend/’, ‘libgo/’
  - ‘libbacktrace/’
  - ‘libffi/’
  - ‘libgrust/’: Rust standard library (upcoming)
  - ‘libiberty/’
  - ‘libsanitizer/’
  - ‘zlib/’
  - GMP, MPFR, MPC, ISL



GCC structure, “binary-level”, and related tools

- ‘gcc’, ‘g++’, ‘gfortran’, … – drivers
- ‘cc1’, ‘cc1plus’, ‘f951’, … – actual compilers, for each source language
- Linker/LTO interfacing
- Provided by GCC installation:
  - libgcc, libatomic, … – general compiler support
  - libstdc++, libgfortran, … – language support/standard library
  - libasan, libgomp, … – feature support
- Provided externally:
  - GNU Binutils (‘as’, ‘ld’, …)
  - GLIBC
  - GDB
  - Etc.



GCC structure, “internally”, greatly simplified

- Language front ends
  - AST
- Several intermediate languages/internal representations
  - GENERIC, GIMPLE (high, low, SSA), RTL (also RTL-SSA)
  - Front end: language-specific/target-agnostic
  - Middle end: language-agnostic/target-agnostic
  - Back end: language-agnostic/target-specific
  - … in theory, at least…
  - In practice: a lot of hooks to retrieve language-specifics, target-specifics
    - What is a C ‘int’? How to implement C++ ‘throw’/’catch’?
  - Code optimizations at all levels
- Target code generation
  - RTL/MD



Legal aspects

- GCC is a GNU project
- Free Software Foundation is the legal entity running the GNU project
- Free Software Foundation holds copyright (with exceptions)

- FSF copyright assignment is not mandatory anymore for GCC contributions: another option is <https://gcc.gnu.org/dco.html> "Developer's Certificate of Origin (DCO) Sign-off"
  - <https://inbox.sourceware.org/gcc/CAGWvnyme6cQUGb+G4=tesNYqLYBSGnDYb95LH2zVUgxovHU7kw@mail.gmail.com> "Update to GCC copyright assignment policy", David Edelsohn (GCC Steering Committee), 2021-06-01



GCC Steering Committee

- <https://gcc.gnu.org/steering.html>
- “intent of preventing any particular individual, group or organization from getting control over the project”
- “primary purpose is to make major decisions in the best interests of the GCC project and to ensure that the project adheres to its fundamental principles found in the project's mission statement”
  - <https://gcc.gnu.org/gccmission.html>
- “appointed by the FSF as the official GNU maintainer for GCC”
- “Membership in the steering committee is a personal membership”



https://gcc.gnu.org/gccmission.html

GCC Development Mission Statement (2021-06-01)

GCC development is a part of the GNU Project, aiming to improve the compiler used in the GNU system including the GNU/Linux variant. The GCC development effort uses an open development environment and supports many other platforms in order to foster a world-class optimizing compiler, to attract a larger team of developers, to ensure that GCC and the GNU system work on multiple architectures and diverse environments, and to more thoroughly test and extend the features of GCC.

Free Software Project

- Supporting the goals of the GNU project.
- Compilers are available under the terms of the GPL.
- Other components (runtime libraries, testsuites, etc) are available under various free licenses.
- Copyrights are held by individual authors or the FSF.
- Patches must be legally acceptable for redistribution under the applicable license.

Design and Development Goals

- New languages
- New optimizations
- New targets
- Improved runtime libraries
- Faster debug cycle
- Various other infrastructure improvements

Open Development Environment

- Encourage cooperation and communication between developers.
- Work more closely with "consumers".
- Code available to everyone at any time, and everyone is welcome to participate in development.
  - Patches will be considered equally based on their technical merits.
  - All individuals and companies are welcome to contribute as long as they accept the ground rules.
- Open mailing lists.
- Developer friendly tools and procedures (i.e. [version control], multiple maintainers, etc).
- Conflicts of interest exist for many GCC developers; the developers as well as the GCC Steering Committee will not allow those conflicts of interest to have an effect on the development of GCC.

For questions related to the use of GCC, please consult these web pages and the GCC manuals. If that fails, the gcc-help@gcc.gnu.org mailing list might



GCC structure, “community”, ‘MAINTAINERS’ file

We’ve covered the GCC Steering Committee – now turning attention to those who “do the actual work”:

- “Random” contributors
  - ..., or: how we all started
- “Write After Approval”
  - Git repository access
- “Area” Maintainers, “Area” Reviewers
  - Languages, targets, generic functionality, infrastructure, “miscellaneous”
- Global Reviewers



GCC structure, “community”, externally (incomplete)

- In close contact with GCC “downstreams”
  - Distributions, individual users
- Participation in relevant committees
  - C, C++, Fortran, etc.
  - OpenACC, OpenMP
- Student projects and research using GCC
- Non-upstreamed/WIP GCC code changes
  - Algol 68 and other front ends
  - Various back ends



GCC contribution process, “just five easy steps”

- <https://gcc.gnu.org/contribute.html>
- Download GCC, modify, test, document, submit ;-)

- Patch submission and review by email (usually)
  - Anyone is encouraged to provide reviews, not just appointed reviewers/maintainers
  - …, in practice, though… :-|
- After approval, the contributor ‘git push’es the commit
  - (In pre-Git times, there was no way to properly preserve authorship in commits)
  - …, or a reviewer/others ‘git push’es, if the contributor indicates: no Git access
  - …, or a maintainer takes over the patch, and eventually ‘git push’es



Project infrastructure (excerpt)

Hosted by <https://sourceware.org/>



- Mailing lists: <https://gcc.gnu.org/lists.html>
  - Also: public inbox: <https://inbox.sourceware.org/>
- IRC channels: <https://gcc.gnu.org/wiki/GCConIRC>
- Bugzilla: <https://gcc.gnu.org/bugzilla/>
- Patchwork: <https://patchwork.sourceware.org/project/gcc/list/>
- CI testing: <https://builder.sourceware.org/buildbot/#/builders?tags=gcc>
  - More CI infrastructure via external parties
- Forge: <https://forge.sourceware.org/gcc>
  - Forgejo instance for experimenting new workflows



Project infrastructure, external (incomplete)

- <https://rust-gcc.github.io/> “GCC Front-End For Rust”
  - <https://github.com/Rust-GCC/gccrs>
  - <https://gcc-rust.zulipchat.com/>
- GNU Fortran
  - <https://gfortran-mm.osuosl.org/> (Mattermost)
- Also, all “code imported from/shared with others” (see before) has external infrastructure
- All companies working on GCC maintain some kind of infrastructure
  - For example, also providing more CI testing to the GCC public
- <https://portal.cfarm.net/>, <https://gcc.gnu.org/wiki/CompileFarm>



Some social media

- <https://fosstodon.org/@gnutools>
- <https://bsky.app/profile/gnutools.bsky.social>
- <https://x.com/gnutools>
- <https://www.facebook.com/groups/gnutoolchain/>





Annual GNU Tools Cauldron (GCC Summit, etc.)



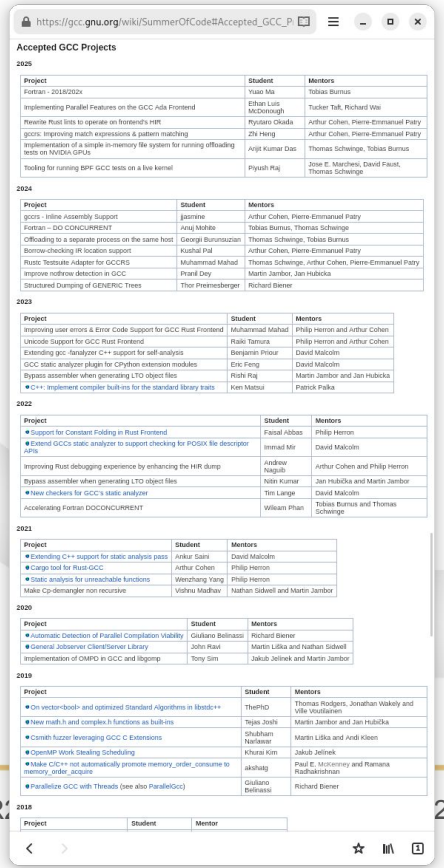
<https://gcc.gnu.org/wiki/cauldron2023>

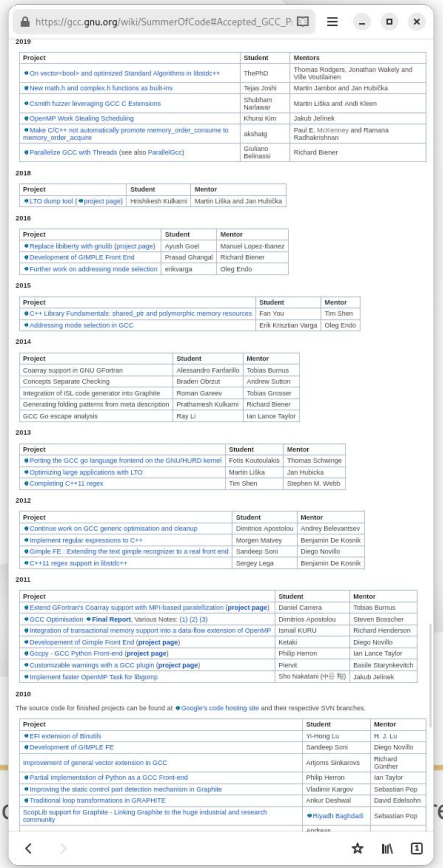
FOSDEM "GCC (GNU Toolchain)" devroom

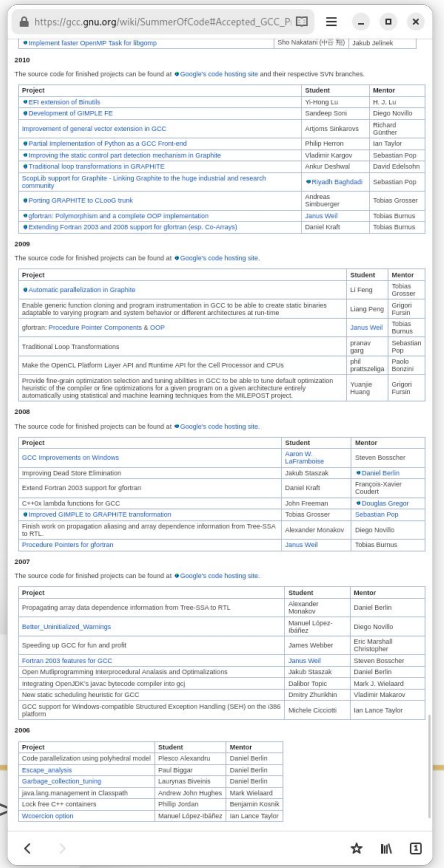


FOSDEM 2024: GCC devroom (the first one ever – instant success: 17 high-quality submissions of diverse topics, out of which we accepted 10, and we had full-room literally all day (~100 people)

Google Summer of Code participation, since 2006







GCC changes

- <https://gcc.gnu.org/gcc-16/changes.html>
  - “Note: GCC 16 has not been released yet, so this document is a work-in-progress.”
- <https://gcc.gnu.org/gcc-15/changes.html>
- Etc.



GCC/Rust: ‘rustc_codegen_gcc’

- <https://github.com/rust-lang/rustc_codegen_gcc>
  “libgccjit AOT codegen for rustc”
- <https://rust-for-linux.com/rustc_codegen_gcc>



GCC/Rust: ‘gccrs’

- <https://blog.rust-lang.org/2024/11/07/gccrs-an-alternative-compiler-for-rust/>
  “gccrs: An alternative compiler for Rust”,
  2024-11-07, Arthur Cohen on behalf of the gccrs project
  - “This is a guest post from the gccrs project, at the invitation of the Rust Project, to clarify the relationship with the Rust Project and the opportunities for collaboration”
- <https://rust-for-linux.com/gccrs>



GCC/Rust: ‘gccrs’

https://rust-gcc.github.io/2025/09/09

As a reminder, our current objective is to get `gccrs` to a state where it can compile the Linux kernel's Rust parts. The Rust-for-Linux project does not depend on the entire Rust standard library (`std`), but only on `core` - like a lot of Rust embedded projects. However, because the project wants to benefit from all of Rust's functionalities, they are reimplementing parts of the standard library using the kernel's memory allocator - this enables Rust kernel module developers to use types such as `Vec<T>`, or `String`. As such, the milestones we worked on this summer are aimed at two objectives:

1. Compile Rust's `core` library, as it is the sole dependency of the Linux kernel's Rust crate
2. Compile the Rust kernel crate

In order to do this, we had to implement some new Rust features over the summer: namely, `try` blocks and `while-let` loops, both used in `core` and within the kernel. The kernel makes use of more recent Rust features that are not present in the version we target, so we must implement them as well - for example, the `offset_of` macro.

Our first objective is the kernel's `alloc` crate, a reimplementation of parts of the Rust `std` crate responsible for implementing dynamically allocated types. Thanks to the work of our contributors over the summer, we



<https://rust-gcc.github.io/2025/09/09/2025-08-monthly-report.html>

GCC/BPF

- <https://gcc.gnu.org/wiki/BPFBackEnd>
- Ask José E. Marchesi, David Faust, or others from Oracle about status

- <https://inbox.sourceware.org/gcc/ri6o6vvxngq.fsf@virgil.suse.cz>
  "Welcome GCC GSoC 2025 participants!"
  - "Piyush Raj will work on tooling for running BPF GCC tests on a live kernel", "mentored by Jose E. Marchesi, David Faust and Thomas Schwinge"
  - *LIVE DEMO!*



Thanks!

